Lab Assignment 13.3 – Code Refactoring Using AI Assistance

Name: B.Sarayu

Roll No: 2303A51842

Task 1: Refactoring – Removing Code Duplication

Refactored Code:

```
def calculate_rectangle(length, width):
    """
    Calculate and return the area and perimeter of a rectangle.
    :param length: Length of the rectangle
    :param width: Width of the rectangle
    :return: (area, perimeter)
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# Rectangle 1
area, perimeter = calculate_rectangle(5, 10)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)

# Rectangle 2
area, perimeter = calculate_rectangle(7, 12)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)

# Rectangle 3
area, perimeter = calculate_rectangle(10, 15)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)
```

Output:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
```

Area of Rectangle: 150
Perimeter of Rectangle: 50

main.py

```python
def calculate_rectangle(length, width):
    """
    Calculate and return the area and perimeter of a rectangle.

    Parameters:
    length (int/float): Length of the rectangle
    width (int/float): Width of the rectangle

    Returns:
    tuple: (area, perimeter)
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter


# Rectangle 1
area, perimeter = calculate_rectangle(5, 10)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)

# Rectangle 2
area, perimeter = calculate_rectangle(7, 12)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)

# Rectangle 3
area, perimeter = calculate_rectangle(10, 15)
print("Area of Rectangle:", area)
print("Perimeter of Rectangle:", perimeter)
```

Output

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50

=== Code Execution Successful ===
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Refactored Code:

```python
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Convert list to set for faster lookup
names_set = set(names)

for name in search_names:
    if name in names_set:
        print(f"{name} is in the list")
    else:
        print(f"{name} is not in the list")
```

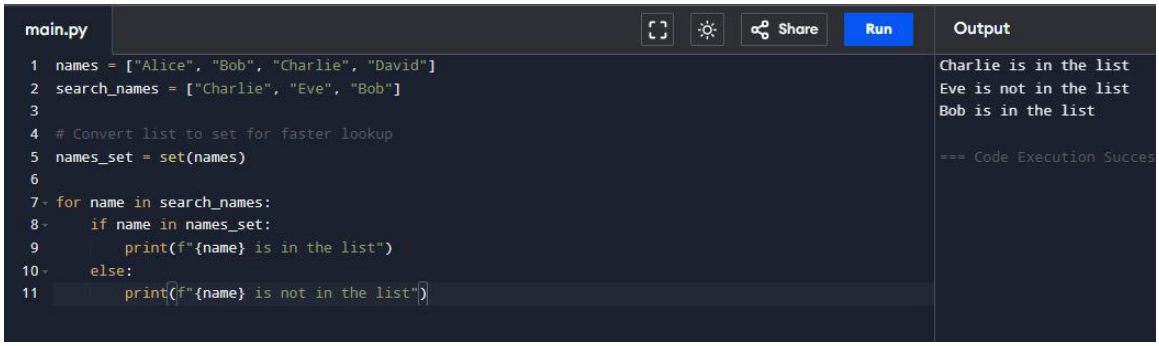
Output:

Charlie is in the list
Eve is not in the list
Bob is in the list

Performance Improvement:

Legacy Code Time Complexity: O(n*m) due to nested loops.

Refactored Code Time Complexity: O(n + m) using set lookup.



Task 3: Refactoring – Extracting Reusable Functions

Refactored Code:

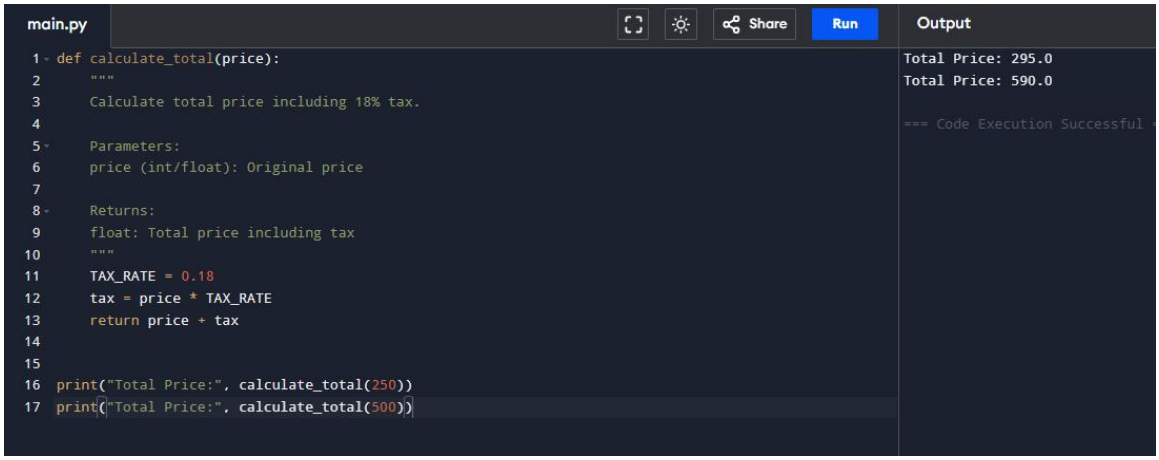
```
def calculate_total(price):
    """
    Calculate total price including 18% tax.
    :param price: Original price
    :return: Total price including tax
    """
    tax_rate = 0.18
    tax = price * tax_rate
    return price + tax

# Example 1
print("Total Price:", calculate_total(250))

# Example 2
print("Total Price:", calculate_total(500))
```

Output:

Total Price: 295.0
Total Price: 590.0



Task 4: Refactoring – Replacing Hardcoded Values with Constants

Refactored Code:

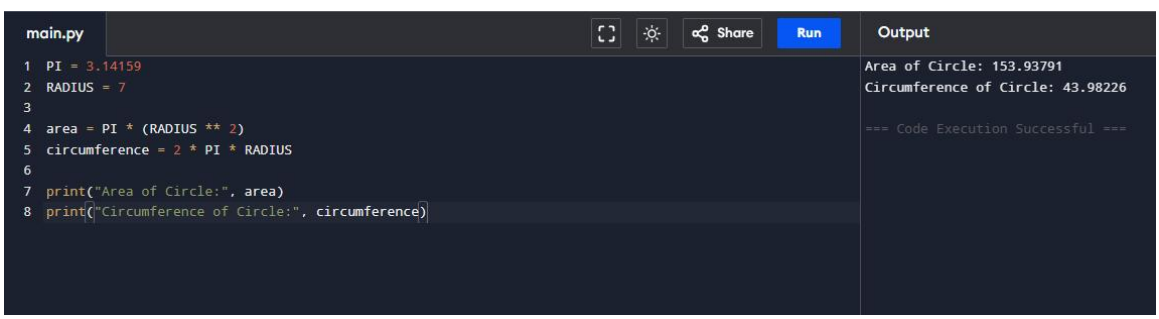
```
PI = 3.14159
RADIUS = 7

area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

Output:

Area of Circle: 153.93891
Circumference of Circle: 43.98226



Task 5: Refactoring – Improving Variable Naming and Readability

Refactored Code:

```
# Base of the triangle
base = 10

# Height of the triangle
```

height = 20

# Calculate area of triangle using formula: (base * height) / 2
area_of_triangle = (base * height) / 2

print(area_of_triangle)

Output:
100.0